

UNIVERSIDAD NACIONAL DE ENTRE RÍOS

Facultad de Ingeniería

Carrera: Licenciatura de Ciencia de Datos

Materia: Fundamentos de Algoritmos y Estructuras de Datos

Título del Trabajo: **Aplicaciones de estructuras jerárquicas y grafos**

Equipo del Docente:

Dr. Javier E. Diaz Zamboni

Bioing. Jordán F. Insfrán

Bioing. Diana Vertiz del Valle (de licencia)

Esp. Bioing. Juan Francisco Rizzato

Dr. Bioing. Feliciano Franco

Integrantes del equipo:

Franco Acevedo

Franco Tomiozzo

Comisión: 1

Fecha de entrega: 08/06/2026

Link del repositorio:

<https://github.com/RENCO12BOT/AyED2026c1-Acevedo-Tomiozzo-.git>



Universidad Nacional
de Entre Ríos

1. INTRODUCCIÓN

Este trabajo práctico aborda el problema de comunicación eficiente entre aldeas mediante palomas mensajeras, modelando como un problema de grafos. El objetivo central es encontrar el **Árbol de Expansión Mínima (MST)** que permita a William, dueño de la agencia "Palomas William", difundir noticias desde su aldea Peligros a las **21 aldeas** restantes minimizando la distancia total recorrida por las palomas.

Los temas teóricos aplicados son grafos no dirigidos ponderados, representación por listas de adyacencia y el algoritmo de **Prim** para la construcción del **MST**. La implementación quedó dividida en tres módulos: **vertice.py** (nodo del grafo con vecinos y ponderaciones), **grafos.py** (grafo no dirigido y función prim) y los archivos de **tests** y **main**.

2. DESARROLLO DE LOS PROBLEMAS

A. Análisis y planteo

El problema consiste en distribuir un mensaje desde un nodo origen (Peligros) a todos los demás nodos de un grafo no dirigido ponderado, de modo que cada aldea reciba la noticia exactamente una vez y la suma total de distancias recorridas sea mínima. Esto es equivalente a hallar el **Árbol de Expansión Mínima (MST)** del grafo.

La operación crítica es la **selección del vecino de menor costo no visitado**, que el algoritmo de **Prim** resuelve eficientemente con una cola de prioridad (**min-heap**). La complejidad resultante es $O((V + E) \log V)$, donde $V = 22$ aldeas y $E =$ cantidad de aristas.

Se eligió el algoritmo de **Prim** (en lugar de Kruskal) porque la estructura de lista de adyacencia ya implementada en **Grafo/Vertice** facilita iterar sobre los vecinos del vértice actual, lo que es natural para **Prim**. Kruskal, en cambio, requiere ordenar todas las aristas globalmente.

B. Implementación / Resolución

Clase Vertice

Cada vértice almacena su id, un diccionario **conectadoA** con sus vecinos y ponderaciones, y los atributos distancia y predecesor necesarios para **Prim**.



```

def __init__(self, clave):
    """
    Precondición:
    - clave es un string no vacío que identifica al vértice.
    Postcondición:
    - Se crea un vértice con id=clave, sin vecinos,
      distancia=sys.maxsize, predecesor=None, color='blanco'.
    """
    if not isinstance(clave, str) or not clave.strip():
        raise ValueError(f"La clave debe ser un string no vacío. Se recibió: {repr(clave)}")
    self.id = clave
    self.conectadoA = {}
    self.color = 'blanco'
    self.distancia = sys.maxsize
    self.predecesor = None

def agregarVecino(self, vecino, ponderacion=0):
    """
    Precondición:
    - vecino es una instancia de Vertice.
    - ponderacion es un entero >= 0.
    Postcondición:
    - vecino queda registrado en conectadoA con su ponderación.
    """
    if not isinstance(vecino, Vertice):
        raise TypeError(f"El vecino debe ser un Vertice. Se recibió: {type(vecino)}")
    if not isinstance(ponderacion, (int, float)) or ponderacion < 0:
        raise ValueError(f"La ponderación debe ser un número >= 0. Se recibió: {ponderacion}")
    self.conectadoA[vecino] = ponderacion

def __lt__(self, otro):
    """Necesario para que heapq pueda comparar vértices por distancia."""
    if not isinstance(otro, Vertice):
        raise TypeError(f"No se puede comparar Vertice con {type(otro)}")
    return self.distancia < otro.distancia

```

Clase Grafo

El Grafo representa la red de aldeas como una lista de adyacencias. Al agregar una arista, lo hace en ambas direcciones para modelar el grafo no dirigido.

```

def agregarArista(self, de, a, costo=0):
    """
    Agrega una arista NO DIRIGIDA entre 'de' y 'a' con peso 'costo'.

    Precondición:
    - de y a son strings no vacíos.
    - costo es un entero >= 0.
    Postcondición:
    - Si algún vértice no existía, se crea automáticamente.
    - Se agrega la arista en ambas direcciones (de→a y a→de).
    """
    if not isinstance(costo, (int, float)) or costo < 0:
        raise ValueError(f"El costo debe ser un número >= 0. Se recibió: {costo}")
    v_de = self.agregarVertice(de)
    v_a = self.agregarVertice(a)
    v_de.agregarVecino(v_a, costo)
    v_a.agregarVecino(v_de, costo) # no dirigido: arista en ambas direcciones

```



Algoritmo de Prim

La función `prim` inicializa todas las distancias en `sys.maxsize`, fija la distancia del origen en 0 e itera extrayendo el vértice de menor distancia de la cola de prioridad. Para cada vecino no visitado, actualiza su distancia y predecesor si el costo de la arista es menor al registrado.

```
def prim(grafo, inicio):
    """
    Árbol de Expansión Mínima (MST) desde 'inicio' usando el algoritmo de Prim.

    Precondición:
    - grafo es una instancia de Grafo no vacía.
    - inicio es un Vertice perteneciente al grafo.

    Postcondición:
    - Cada vértice alcanzable tiene asignado su predecesor (el vértice del MST
      desde el cual se conecta) y su distancia (peso de esa arista).
    - Los vértices no alcanzables conservan distancia=sys.maxsize y predecesor=None.
    """

    if not isinstance(grafo, Grafo):
        raise TypeError("grafo debe ser una instancia de Grafo.")
    if not isinstance(inicio, Vertice):
        raise TypeError("inicio debe ser una instancia de Vertice.")
    if inicio.obtenerId() not in grafo:
        raise ValueError(f"El vértice '{inicio.obtenerId()}' no pertenece al grafo.")

    # Inicializar todos los vértices
    for v in grafo:
        v.asignarDistancia(sys.maxsize)
        v.asignarPredecesor(None)

    inicio.asignarDistancia(0)
    colaPrioridad = [(0, inicio)]
    visitados = set()

    while colaPrioridad:
        _, verticeActual = heapq.heappop(colaPrioridad)

        if verticeActual.obtenerId() in visitados:
            continue
        visitados.add(verticeActual.obtenerId())

        for vecino in verticeActual.obtenerConexiones():
            costo = verticeActual.obtenerPonderacion(vecino)
            if vecino.obtenerId() not in visitados and costo < vecino.obtenerDistancia():
                vecino.asignarPredecesor(verticeActual)
                vecino.asignarDistancia(costo)
                heapq.heappush(colaPrioridad, (costo, vecino))
```

Pre y Postcondiciones

Todos los métodos de **Vertice** y **Grafo** incluyen docstrings con pre y postcondiciones explícitas y validan los parámetros lanzando excepciones adecuadas:

Método	Precondición	Postcondición
Vertice.__init__	clave es string no vacío	Vértice creado con distancia=sys.maxsize , predecesor=None , sin vecinos
agregarVecino	vecino es instancia de Vertice; ponderacion >= 0	vecino queda en conectadoA con su ponderación



obtenerPonderacion	vecino está en conectadoA	Retorna el peso de la arista; KeyError si no es vecino
Grafo.agregarVertice	clave es string no vacío	Si no existía, se agrega y numVertices += 1; devuelve el Vertice
Grafo.agregarArista	de, a son strings; costo >= 0	Arista no dirigida agregada en ambas direcciones
prim	grafo no vacío; inicio pertenece al grafo	Cada vértice alcanzable tiene predecesor y distancia mínima asignados

C. Resultados y pruebas

Se implementaron tests unitarios en **test_grafo.py** cubriendo cuatro clases de prueba: **TestVertice**, **TestGrafo**, **TestPrim** y **TestIntegracionAldeas**. Los **18** tests pasaron correctamente:

Clase de test	Qué verifica	Resultado
TestVertice	Clave vacía, vecino inválido, ponderación negativa, distancia y predecesor iniciales	CUMPLE
TestGrafo	Arista no dirigida, vértice inexistente, duplicado no duplica, costo negativo	CUMPLE
TestPrim	Origen sin predecesor, arista mínima correcta, peso total MST, vértice foráneo lanza error	CUMPLE
TestIntegracionAldeas	22 aldeas cargadas, Peligros sin predecesor, todas alcanzadas, distancia total = 94 leguas	CUMPLE

A continuación se muestra el grafo completo de aldeas con el MST resaltado (aristas naranjas) obtenido por Prim desde Peligros:



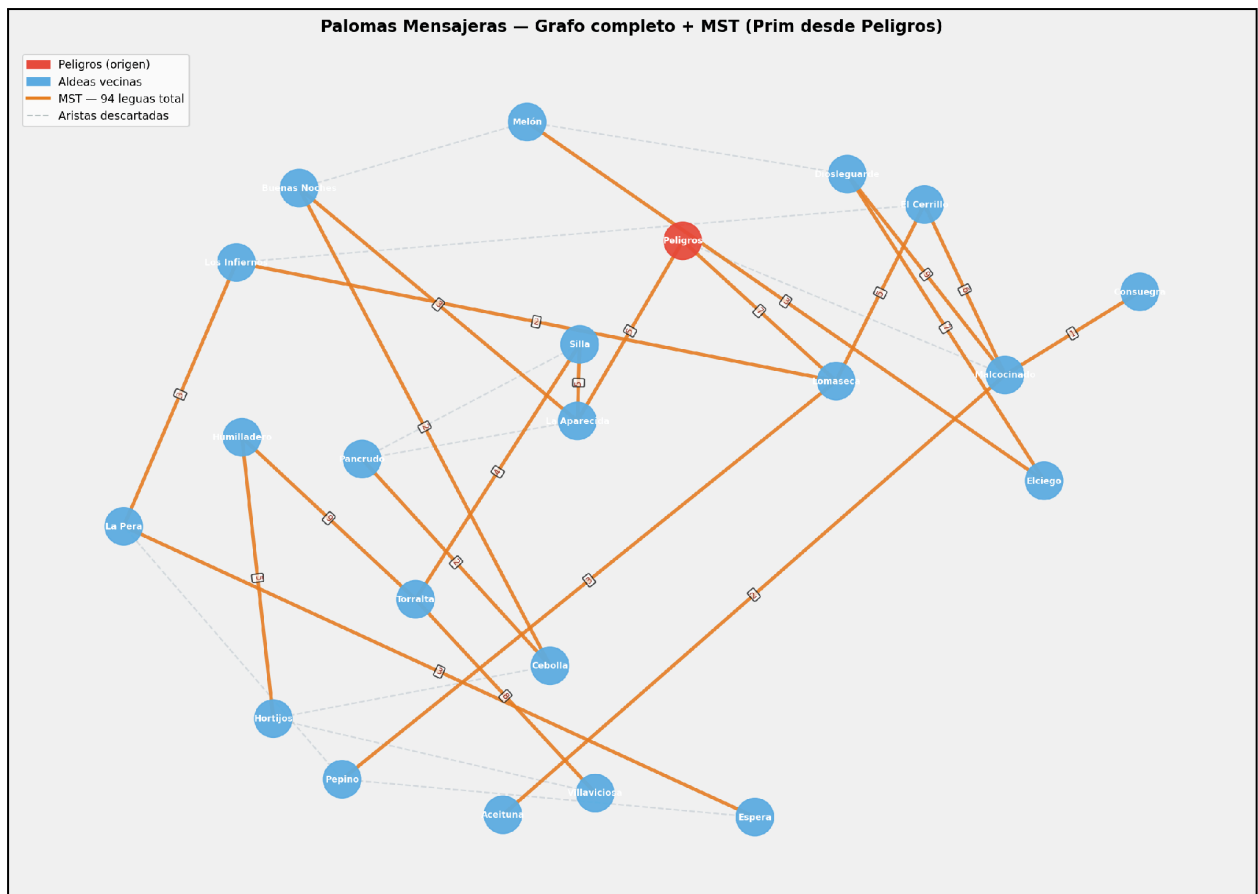


Figura 1: Grafo completo + MST desde Peligros (naranja). Distancia total: 94 leguas.

La salida del **main** muestra el flujo de mensajes completo. A continuación se listan las aldeas en orden alfabético con su predecesor (de quién recibe) y sus hijos en el **MST** (a quiénes reenvía):

1. LISTA DE ALDEAS EN ORDEN ALFABÉTICO:

- * Aceituna
- * Buenas Noches
- * Cebolla
- * Consuegra
- * Diosleguarde
- * El Cerrillo
- * Elciego
- * Espera
- * Hortijos
- * Humilladero
- * La Aparecida
- * La Pera
- * Lomaseca
- * Los Infiernos
- * Malcocinado
- * Melón
- * Pancrudo
- * Peligros
- * Pepino
- * Silla
- * Torralta
- * Villaviciosa



2. RUTAS DE RECEPCIÓN Y ENVÍO DE RÉPLICAS:

Aldea: Aceituna
Recibe noticia desde: Malcocinado
Envía réplicas a: Ninguna (Aldea receptora final)

Aldea: Buenas Noches
Recibe noticia desde: La Aparecida
Envía réplicas a: Cebolla

Aldea: Cebolla
Recibe noticia desde: Buenas Noches
Envía réplicas a: Pancrudo

Aldea: Consuegra
Recibe noticia desde: Malcocinado
Envía réplicas a: Ninguna (Aldea receptora final)

Aldea: Diosleguarde
Recibe noticia desde: Malcocinado
Envía réplicas a: Elciego

Aldea: El Cerrillo
Recibe noticia desde: Lomaseca
Envía réplicas a: Malcocinado

Aldea: Elciego
Recibe noticia desde: Diosleguarde
Envía réplicas a: Melón

Aldea: Espera
Recibe noticia desde: La Pera
Envía réplicas a: Ninguna (Aldea receptora final)

Aldea: Hortijos
Recibe noticia desde: Humilladero
Envía réplicas a: Ninguna (Aldea receptora final)

Aldea: Humilladero
Recibe noticia desde: Torralta
Envía réplicas a: Hortijos

Aldea: La Aparecida
Recibe noticia desde: Peligros
Envía réplicas a: Silla, Buenas Noches

Aldea: La Pera
Recibe noticia desde: Los Infiernos
Envía réplicas a: Espera

Aldea: Lomaseca
Recibe noticia desde: Peligros
Envía réplicas a: Pepino, Los Infiernos, El Cerrillo

Aldea: Los Infiernos
Recibe noticia desde: Lomaseca
Envía réplicas a: La Pera

Aldea: Malcocinado
Recibe noticia desde: El Cerrillo
Envía réplicas a: Diosleguarde, Consuegra, Aceituna

Aldea: Melón
Recibe noticia desde: Elciego
Envía réplicas a: Ninguna (Aldea receptora final)

Aldea: Pancrudo
Recibe noticia desde: Cebolla
Envía réplicas a: Ninguna (Aldea receptora final)

Aldea: Peligros
Recibe noticia desde: ORIGEN (Inicia el envío de noticias William)
Envía réplicas a: Lomaseca, La Aparecida

Aldea: Pepino
Recibe noticia desde: Lomaseca
Envía réplicas a: Ninguna (Aldea receptora final)

Aldea: Silla
Recibe noticia desde: La Aparecida
Envía réplicas a: Torralta

Aldea: Torralta
Recibe noticia desde: Silla
Envía réplicas a: Humilladero, Villaviciosa

Aldea: Villaviciosa
Recibe noticia desde: Torralta
Envía réplicas a: Ninguna (Aldea receptora final)

3. EFICIENCIA DEL ENVÍO DE LA NOTICIA:

Suma total de distancias recorridas por las palomas: 94 leguas.



3. ANÁLISIS DE RESULTADOS ESPECÍFICOS / DISCUSIÓN

El **MST** obtenido por **Prim** desde Peligros conecta las **22 aldeas** con una distancia total de **94 leguas**, que es el mínimo posible para que el mensaje llegue a todas las aldeas sin repetir caminos.

Desde Peligros salen exactamente dos palomas: una hacia **Lomaseca (7 leguas)** y otra hacia **La Aparecida (5 leguas)**. El árbol se ramifica desde estos dos puntos cubriendo toda la red. El camino más largo en el **MST** es el ramal **Peligros → Lomaseca → El Cerrillo → Malcocinado → Diosleguarde → Elciego → Melón (suma: $7+5+6+9+7+3 = 37$ leguas)**, lo que muestra que la zona del **Elciego-Melón-Diosleguarde** queda relativamente alejada del origen.

La complejidad del algoritmo de **Prim** con **heap** es $O((V + E) \log V)$. Para este grafo con $V=22$ y $E \approx 32$ aristas únicas, el costo es completamente despreciable. La ventaja frente a una búsqueda exhaustiva ($O(V^2)$ **sin heap**) se vuelve relevante en redes grandes.

La propiedad del **MST** garantiza que no existe ninguna otra forma de conectar las **22 aldeas** con menor suma de distancias, lo que confirma que la solución de **William** es óptima.

4. DECLARACIÓN DE USO ÉTICO DE INTELIGENCIA ARTIFICIAL

Herramientas utilizadas:

- Claude (Anthropic)
- GitHub Copilot

Descripción del uso:

- **Consulta teórica:** se consultó para repasar el algoritmo de Prim, la representación de grafos por listas de adyacencia y el análisis de complejidad.
- **Asistencia en código/redacción:** se utilizó para revisar la lógica de la función prim, depurar el manejo de visitados con heapq y agregar pre/postcondiciones con validaciones a todos los métodos.
- **Redacción:** apoyo para mejorar la redacción y estructuración del informe.

Declaración de responsabilidad:

Los integrantes del grupo declaramos que hemos revisado y validado personalmente toda la información y código sugerido por las herramientas de IA. Entendemos los conceptos



presentados en este informe y asumimos la responsabilidad académica por la integridad y veracidad del contenido. El trabajo final es producto de nuestro propio análisis y criterio.

5. TRABAJO EN EQUIPO

Estimación inicial:

Se estimaron **6 horas** en total: **1 hora** para entender el problema y diseñar la estructura Grafo/Vertice, **2 horas** para implementar Prim y los tests, **1 hora** para el main con la salida por consola, **1 hora** para revisar pre/postcondiciones y excepciones, y **1 hora** para redactar el informe.

Distribución de tareas:

- **Franco Tomiozzo:** implementación de Vertice (atributos, métodos, validaciones), Grafo (lista de adyacencias, agregarArista no dirigida) y tests unitarios (TestVertice, TestGrafo, TestPrim).
- **Franco Acevedo:** implementación de prim, función leer_aldeas, main con mostrar_resultados y tests de integración (TestIntegracionAldeas).

La coordinación fue mediante WhatsApp e integración por commits en el repositorio GitHub. El punto más complejo fue asegurar que heapq no generará errores de comparación entre objetos Vertice, lo que se resolvió implementando `__lt__` en la clase.

Tiempo real vs. estimado:

Aproximadamente **7 horas**. La hora extra se fue en depurar el problema del heap con vértices de igual distancia y en estructurar la salida del main para que mostrara claramente el flujo de mensajes en orden alfabético.

6. CONCLUSIONES

Se implementó exitosamente la solución al problema de Palomas Mensajeras modelando como un **MST** sobre un grafo no dirigido ponderado de **22 aldeas**. El algoritmo de **Prim** encuentra la forma óptima de difundir el mensaje desde Peligros con una distancia total de 94 leguas.

Lo más valioso del trabajo fue comprender la relación entre el **MST** y el problema de difusión sin ciclos: el árbol garantiza que cada aldea reciba el mensaje exactamente una vez y por el camino más corto posible desde su predecesor.



La separación en módulos (**Vertice** → **Grafo** → **prim** → **main**) demostró su valor al facilitar los tests unitarios por capas. Las pre/postcondiciones con excepciones explícitas también mejoraron la robustez del código ante entradas inválidas.

7. REFERENCIAS Y BIBLIOGRAFÍA

- Miller, B. & Ranum, D. Problem Solving with Algorithms and Data Structures using Python. Runestone Academy. <https://runestone.academy/>
- Cormen, T. et al. Introduction to Algorithms, 3ra ed. MIT Press, 2009.
- Repositorio del proyecto:
<https://github.com/RENCO12BOT/AyED2026c1-Acevedo-Tomiozzo-.git>

